

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

کنترل BRAM با FSM

کنترل حافظه *BRAM* تک پورت با استفاده از ماشین حالت محدود
(*FSM*) در *VHDL*

درس : *VHDL* 

استاد : سرکار خانم دکتر سلیمی 

آذر ماه ۱۴۰۴ 

دانشگاه آزاد اسلامی واحد اصفهان 

تهیه و تنظیم : یاسین سموعی



FSM چیست؟ (Finite State Machine)

◊ تعریف ساده:

یک ماشین حالت محدود مثل یک دستگاه با چندین "حالت" مشخص است که فقط در یک لحظه در یکی از این حالت‌ها قرار دارد و با دریافت دستور، از یک حالت به حالت دیگر می‌رود.

مثال روزمره: چراغ راهنمایی

یک چراغ راهنمایی ساده یک **FSM** است:

States: حالت‌ها

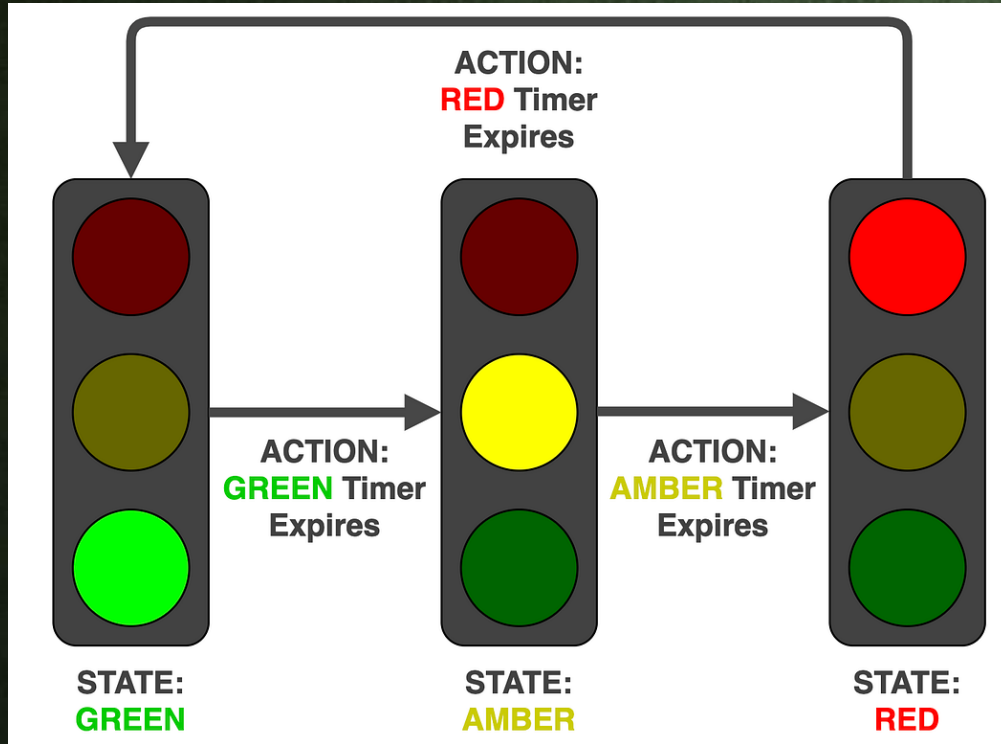
قرمز (اتومبیل‌ها بایستند) ●

سبز (اتومبیل‌ها حرکت کنند) ●

زرد (آماده توقف) ●

ورودی/دستور : گذشت زمان (مثلاً تایمر ۳۰ ثانیه‌ای).

انواع FSM



قوانین گذر حالت (Transition Rules):

اگر در حالت قرمز هستی و ۳۰ ثانیه گذشت، به حالت سبز برو.

اگر در حالت سبز هستی و ۳۰ ثانیه گذشت، به حالت زرد برو.

اگر در حالت زرد هستی و ۵ ثانیه گذشت، به حالت قرمز برو.

خروجی (Output): همان رنگ نور است که می بینیم.

نوع	تعریف	مثال در پروژه کنترل حافظه
Moore مور	خروجی فقط به حالت فعلی بستگی دارد.	data_valid فقط در حالت READ_DONE فعال می شود.
Mealy میلی	خروجی به حالت فعلی و ورودی بستگی دارد.	(در این طراحی استفاده نشده است)

FSM مور (Moore)

ویژگی	توضیح
تعریف	خروجی فقط به حالت فعلی Current State بستگی دارد.
بلوک‌دیagram	ورودی → منطق حالت بعدی → رجیستر حالت → منطق خروجی → خروجی
زمان تغییر خروجی	خروجی در ابتدای سیکل کلاک بعدی (پس از به‌روزرسانی حالت) تغییر می‌کند.
مثال در پروژه من	سیگنال data_valid فقط زمانی '1' می‌شود که FSM در حالت READ_DONE قرار گرفته باشد. ورودی cmd در این لحظه دیگر تأثیری روی این خروجی ندارد.
مثال روزمره	چراغ راهنمایی: رنگ نور (خروجی) فقط به حالت فعلی (قرمز/سبز/زرد) بستگی دارد، نه به اینکه دکمه‌ی عابر پیاده فشرده شده باشد یا نه.
مزایا	• پایداری بیشتر: خروجی مستقل از ورودی، نویزپذیری کمتر. • تأخیر خروجی قابل پیش‌بینی (یک سیکل بعد از تغییر حالت).
معایب	• ممکن است برای تولید برخی خروجی‌ها به حالت‌های بیشتر نیاز داشته باشد. • واکنش کندتر: خروجی تا به‌روزرسانی حالت تغییر نمی‌کند.

FSM میلی (Mealy)

ویژگی	توضیح
تعریف	خروجی به حالت فعلی Current State و ورودی فعلی Current Input بستگی دارد.
بلوک‌دیagram	ورودی → منطق حالت بعدی → رجیستر حالت و ورودی + حالت فعلی → منطق خروجی → خروجی
زمان تغییر خروجی	خروجی بلافاصله پس از تغییر ورودی (حتی در وسط سیکل کلاک) می‌تواند تغییر کند.
مثال فرضی در پروژه	اگر data_valid در همان سیکلی که دستور خواندن ("01" cmd) در حالت IDLE دریافت می‌شود، فعال می‌شود، یک FSM Mealy بود. (در طراحی ما این‌گونه نیست).
مثال روزمره	درب اتوماتیک فروشگاه: موتور (خروجی) هم به حالت فعلی (باز/بسته) و هم به سنسور حرکت (ورودی) بستگی دارد. اگر در حالت "بسته" باشی و سنسور حرکت را تشخیص دهد، در باز می‌شود.
مزایا	• واکنش سریعتر: خروجی بدون انتظار برای تغییر حالت، به ورودی پاسخ می‌دهد. • نیاز به حالت‌های کمتر: برای توصیف یک رفتار مشابه اغلب حالت‌های کمتری نسبت به Moore نیاز دارد.
معایب	• پیچیدگی تحلیل زمانی: خروجی‌ها می‌توانند بین سیکل‌های کلاک تغییر کنند. • حساسیت به نویز ورودی: تغییرات ناخواسته در ورودی می‌تواند خروجی ناخواسته ایجاد کند.

مثال عملی انواع FSM

🔍 تحلیل ساده:

موتور هم به حالت در بستگی دارد هم به سنسور حرکت
با تغییر سنسور، موتور همان لحظه فعال می‌شود

موتور	حالت بعدی	سنسور	حالت فعلی
0	CLOSED	0	CLOSED
1	OPEN	1	CLOSED
0	OPEN	1	OPEN
1	CLOSED	0	OPEN

🔴 مثال ۱: ماشین حالت **Mealy**

📱 درب اتوماتیک فروشگاه

ورودی:

Sensor سنسور حرکت

`0` = کسی نیست

`1` = شخص نزدیک شده

خروجی:

Motor موتور درب

`0` = خاموش

`1` = روشن (باز یا بسته شدن در)

حالت‌ها:

CLOSED در بسته

OPEN در باز

مثال عملی انواع FSM

تحلیل ساده:

● مثال ۲: ماشین حالت ***Moore

🚦 چراغ راهنمایی

ورودی:

رنگ چراغ فقط به حالت فعلی بستگی دارد

ورودی تایمر فقط تعیین می کند کی حالت عوض شود

خروجی وسط حالت تغییر نمی کند

Timer پایان زمان هر چراغ

`۰` = زمان تمام نشده

`۱` = زمان تمام شده

خروجی:

Light رنگ چراغ

حالت ها:

RED

GREEN

YELLOW

حالت فعلی	Timer	حالت بعدی	چراغ
RED	0	RED	قرمز
RED	1	GREEN	قرمز
GREEN	0	GREEN	سبز
GREEN	1	YELLOW	سبز
YELLOW	0	YELLOW	زرد
YELLOW	1	RED	زرد

Block RAM (BRAM) در FPGA

◇ تعریف:

Block RAM (BRAM) یک منبع حافظه داخلی اختصاصی، با دسترسی تصادفی و همزمان (**Synchronous**)

است که در تراشه‌های **FPGA** تعبیه شده است. این حافظه برای ذخیره‌سازی حجم متوسطی از داده با کارایی و سرعت بالا طراحی شده است.

صرفه‌جویی در منابع: استفاده از منابع حافظه‌ی اختصاصی **FPGA**، باعث آزاد شدن سلول‌های منطقی **LUT** برای توابع دیگر می‌شود.

قابلیت پیکربندی: امکان تنظیم عرض داده (**Data Width**) و عمق آدرس (**Address Depth**) متناسب با نیاز طراحی.

BRAM در پروژه کنونی:

در این پروژه از یک **BRAM Single-Port** با مشخصات زیر استفاده شده است:

پیکربندی: ۸ بیت عرض داده $\times 2^8 = 256$ محل حافظه

Latency: ۱ سیکل کلاک (به دلیل ثبت خروجی در **dout_reg**)

هدف: شبیه‌سازی و کنترل یک حافظه پایه برای درک تعامل **FSM** و واحد حافظه

انواع معماری‌های Block RAM

۱. **BRAM** تک‌پورته (**Single-Port**)

این نوع، ساده‌ترین معماری حافظه است که تنها دارای یک پورت دسترسی می‌باشد. این پورت واحد در هر سیکل کلاک تنها قادر به انجام یک عمل خواندن یا یک عمل نوشتن است. به عبارت دیگر، امکان انجام همزمان دو عملیات بر روی حافظه وجود ندارد. این محدودیت نیازمند یک کنترلر ترتیبی (مانند **FSM** طراحی‌شده در این پروژه) است تا از تداخل دستورات جلوگیری نماید. معماری تک‌پورت به دلیل سادگی، کمترین منابع سخت‌افزاری را مصرف می‌کند و برای کاربردهایی که دسترسی‌های پشت‌سرهم و غیرهمزمان کافی است، گزینه مناسبی محسوب می‌شود.

۲. **BRAM** دوپورته (**Dual-Port**)

این معماری پیشرفته‌تر، دارای دو پورت مجزای دسترسی (پورت **A**) و (پورت **B**) است. ویژگی اصلی این حافظه، امکان دسترسی همزمان دو ماژول مستقل به آرایه حافظه واحد می‌باشد. هر پورت آدرس، داده و سیگنال کنترل مخصوص به خود را دارد. نکته حائز اهمیت این است که در شرایط خاص، مانند نوشتن همزمان دو پورت در یک آدرس واحد، رفتار حافظه باید به دقت مدیریت و تعریف شود. **BRAM** دوپورته برای سیستم‌هایی که نیاز به اشتراک‌گذاری داده با پهنای‌بند بالا بین دو واحد پردازشی دارند (مانند پردازش خط لوله‌ای یا سیستم‌های چند هسته‌ای ساده) ایده‌آل است.

کد Block RAM بخش اول

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.numeric_std.all;
```

```
entity bram_sram is
```

```
  generic(  
    ADDR_WIDTH : integer := 8;  
    DATA_WIDTH : integer := 8  
  );
```

```
  port(  
    clk : in std_logic;
```

```
    rst : in std_logic;
```

```
    we : in std_logic; -- '1' => write
```

```
    addr : in std_logic_vector(ADDR_WIDTH-1 downto 0);
```

```
    din : in std_logic_vector(DATA_WIDTH-1 downto 0);
```

```
    dout : out std_logic_vector(DATA_WIDTH-1 downto 0)
```

```
  );
```

```
end entity;
```

```
architecture rtl of bram_sram is
```

```
  type ram_t is array (0 to (2**ADDR_WIDTH)-1) of std_logic_vector(DATA_WIDTH-1 downto 0);
```

```
  signal mem : ram_t := (others => (others => '0'));
```

```
  signal addr_reg : std_logic_vector(ADDR_WIDTH-1 downto 0) := (others => '0');
```

```
  signal dout_reg : std_logic_vector(DATA_WIDTH-1 downto 0) := (others => '0');
```

کد Block RAM بخش دوم

```
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        -- Initialize all memory to zero
        for i in 0 to (2**ADDR_WIDTH)-1 loop
          mem(i) <= (others => '0');
        end loop;
        addr_reg <= (others => '0');
        dout_reg <= (others => '0');
      else
        -- Register address for read (1 cycle latency)
        addr_reg <= addr;

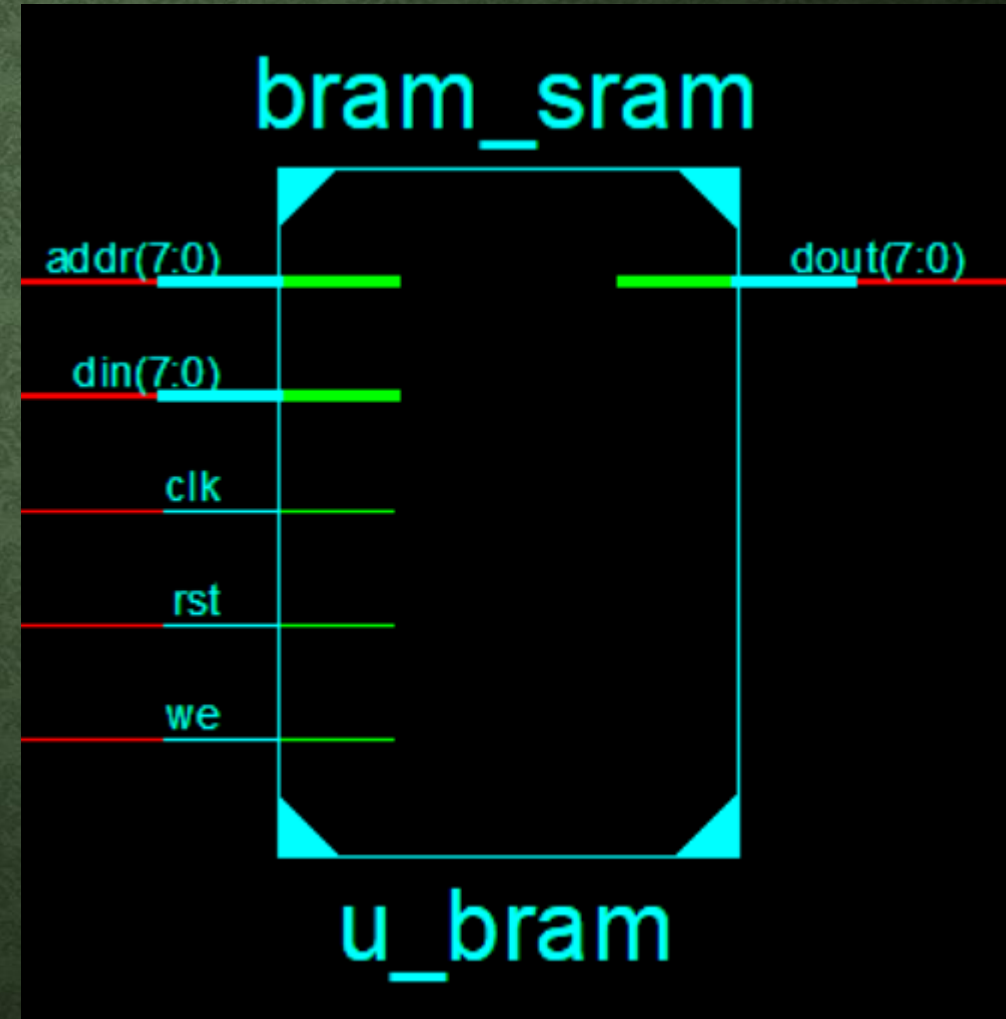
        -- Write operation
        if we = '1' then
          mem(to_integer(unsigned(addr))) <= din;
        end if;

        -- Read operation (with 1 cycle latency)
        dout_reg <= mem(to_integer(unsigned(addr_reg)));
      end if;
    end if;
  end process;

  dout <= dout_reg;
```

end architecture;

یاسین سموعی



کد FSM بخش اول

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity bram_controller is
generic(
  ADDR_WIDTH : integer := 8;
  DATA_WIDTH : integer := 8
);
port(
  clk      : in std_logic;
  rst      : in std_logic;
  cmd      : in std_logic_vector(1 downto 0);
  cmd_valid : in std_logic;
  addr_in  : in std_logic_vector(ADDR_WIDTH-1 downto 0);
  data_in  : in std_logic_vector(DATA_WIDTH-1 downto 0);
  bram_dout : in std_logic_vector(DATA_WIDTH-1 downto 0);
  bram_we   : out std_logic;
  bram_addr : out std_logic_vector(ADDR_WIDTH-1 downto 0);
  bram_din  : out std_logic_vector(DATA_WIDTH-1 downto 0);
  data_out  : out std_logic_vector(DATA_WIDTH-1 downto 0);
  data_valid : out std_logic;
  busy      : out std_logic
);
end entity;
```

```
architecture rtl of bram_controller is
  type state_t is (IDLE, READ_WAIT1, READ_WAIT2, READ_DONE, WRITE_DO, WRITE_DONE);
  signal state : state_t := IDLE;
  signal addr_reg : std_logic_vector(ADDR_WIDTH-1 downto 0) := (others => '0');
  signal data_reg : std_logic_vector(DATA_WIDTH-1 downto 0) := (others => '0');
  signal data_out_reg : std_logic_vector(DATA_WIDTH-1 downto 0) := (others => '0');
  signal bram_we_reg : std_logic := '0';
  signal bram_addr_reg : std_logic_vector(ADDR_WIDTH-1 downto 0) := (others => '0');
  signal bram_din_reg : std_logic_vector(DATA_WIDTH-1 downto 0) := (others => '0');
```

کد FSM بخش دوم

```
begin
  -- Registered outputs
  bram_we  <= bram_we_reg;
  bram_addr <= bram_addr_reg;
  bram_din  <= bram_din_reg;
  data_out  <= data_out_reg;

  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        -- Reset all registers
        state  <= IDLE;
        addr_reg  <= (others => '0');
        data_reg  <= (others => '0');
        data_out_reg <= (others => '0');
        bram_we_reg <= '0';
        bram_addr_reg <= (others => '0');
        bram_din_reg <= (others => '0');
        data_valid <= '0';
        busy      <= '0';
      else
        -- Default values
        bram_we_reg <= '0';
        data_valid <= '0';
```

```
      case state is
        when IDLE =>
          busy <= '0';

          if cmd_valid = '1' then
            busy <= '1';
            addr_reg <= addr_in;
            data_reg <= data_in;

            case cmd is
              when "01" => -- Read command
                -- Apply address to BRAM immediately
                bram_addr_reg <= addr_in;
                state <= READ_WAIT1;

              when "10" => -- Write command
                -- Apply address and data to BRAM immediately
                bram_addr_reg <= addr_in;
                bram_din_reg <= data_in;
                bram_we_reg <= '1'; -- Activate write enable
                state <= WRITE_DO;

              when others => -- Invalid command
                state <= IDLE;
                busy <= '0';
            end case;
          end if;
        end if;
```

کد FSM بخش سوم

```

when READ_WAIT1 =>
    -- First wait state for BRAM read latency
    bram_addr_reg <= addr_reg;
    state <= READ_WAIT2;

when READ_WAIT2 =>
    -- Second wait state (BRAM has 1 cycle latency + we need one more)
    bram_addr_reg <= addr_reg;
    state <= READ_DONE;

when READ_DONE =>
    -- Capture data from BRAM
    data_out_reg <= bram_dout;
    data_valid <= '1';
    state <= IDLE;

when WRITE_DO =>
    -- Write was performed, keep we active for one cycle
    bram_addr_reg <= addr_reg;
    bram_din_reg <= data_reg;
    bram_we_reg <= '1';
    state <= WRITE_DONE;

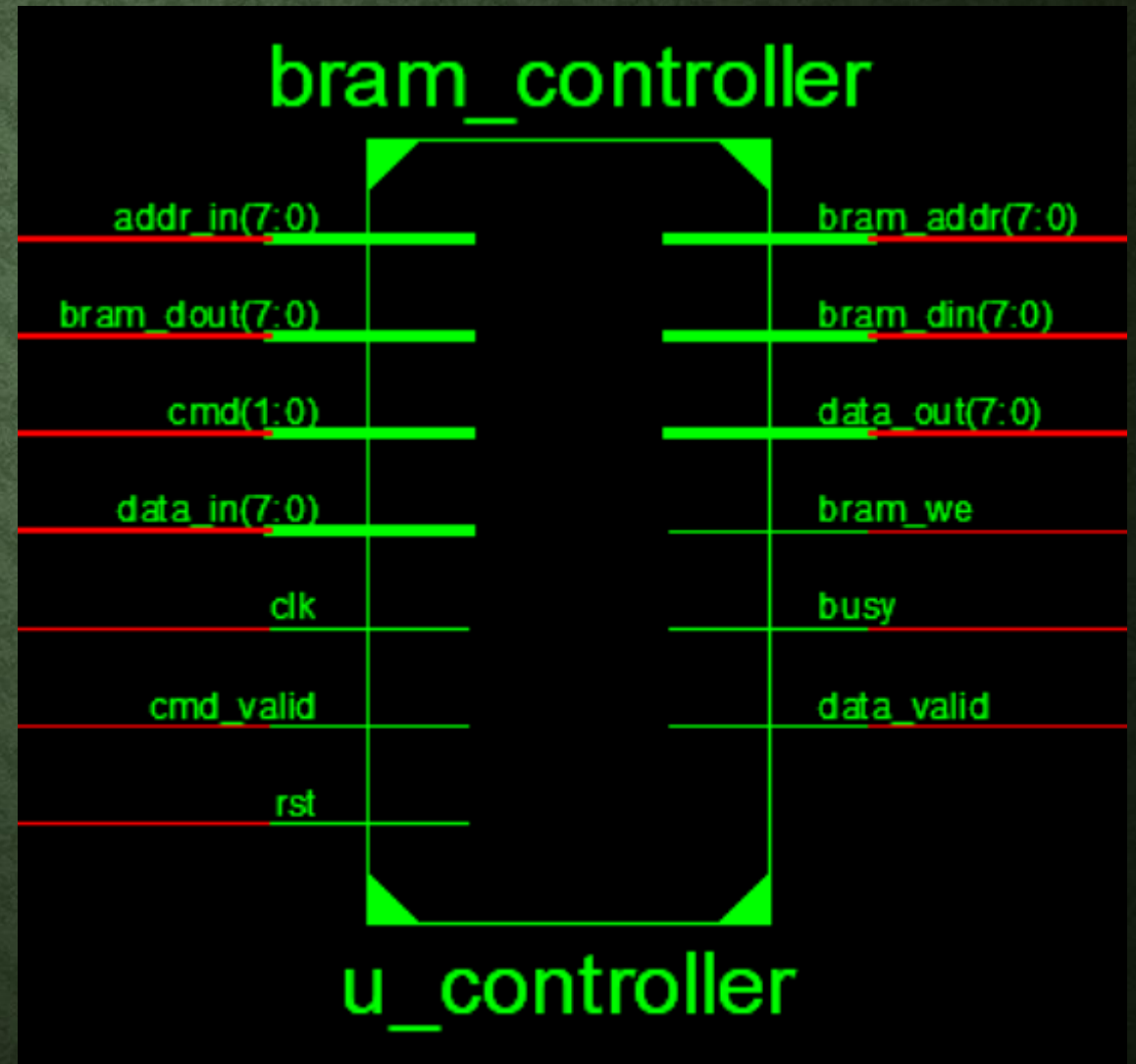
when WRITE_DONE =>
    -- Finish write operation
    bram_we_reg <= '0';
    state <= IDLE;

end case;
end if;
end if;
end process;

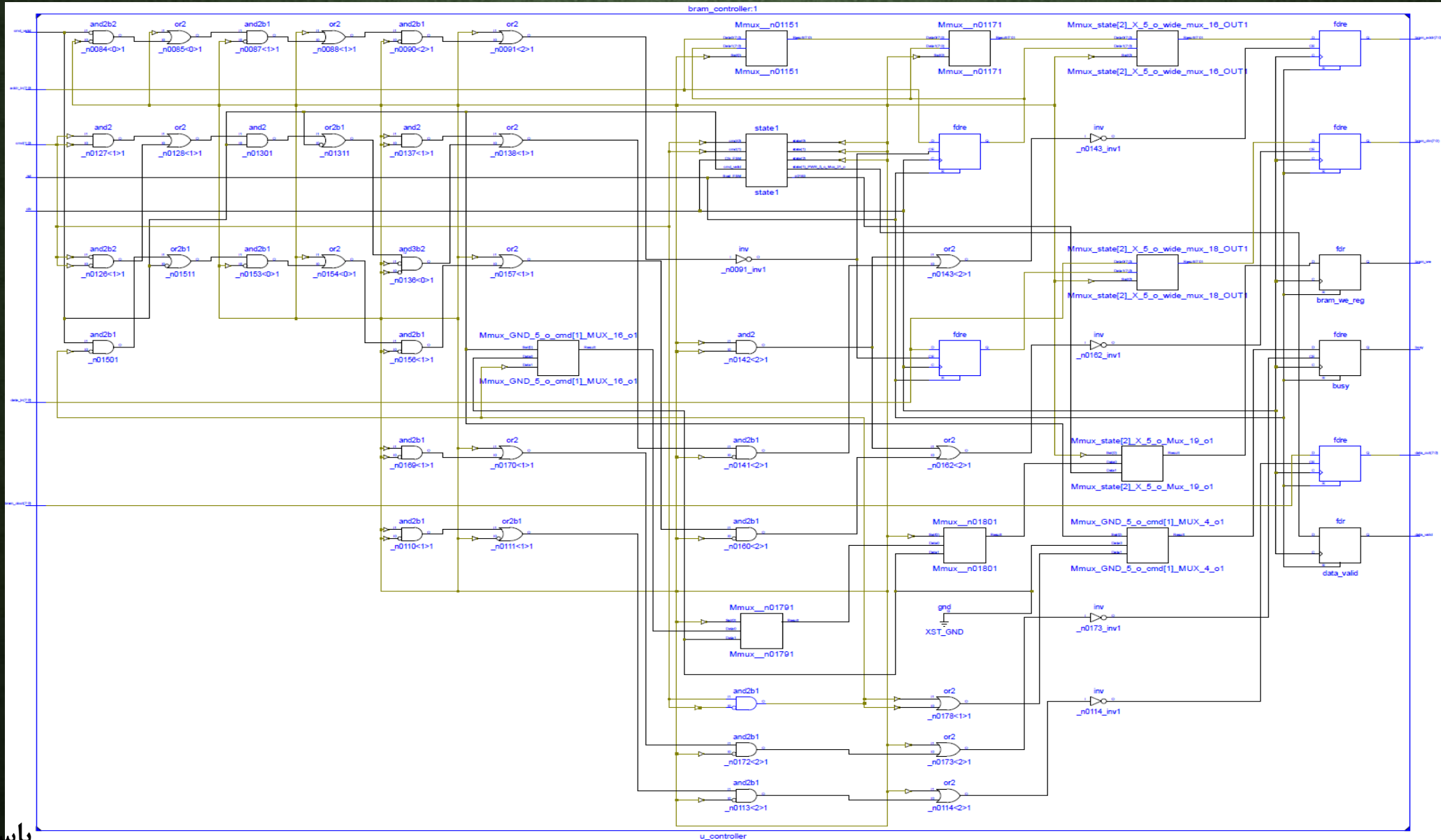
```

end architecture;

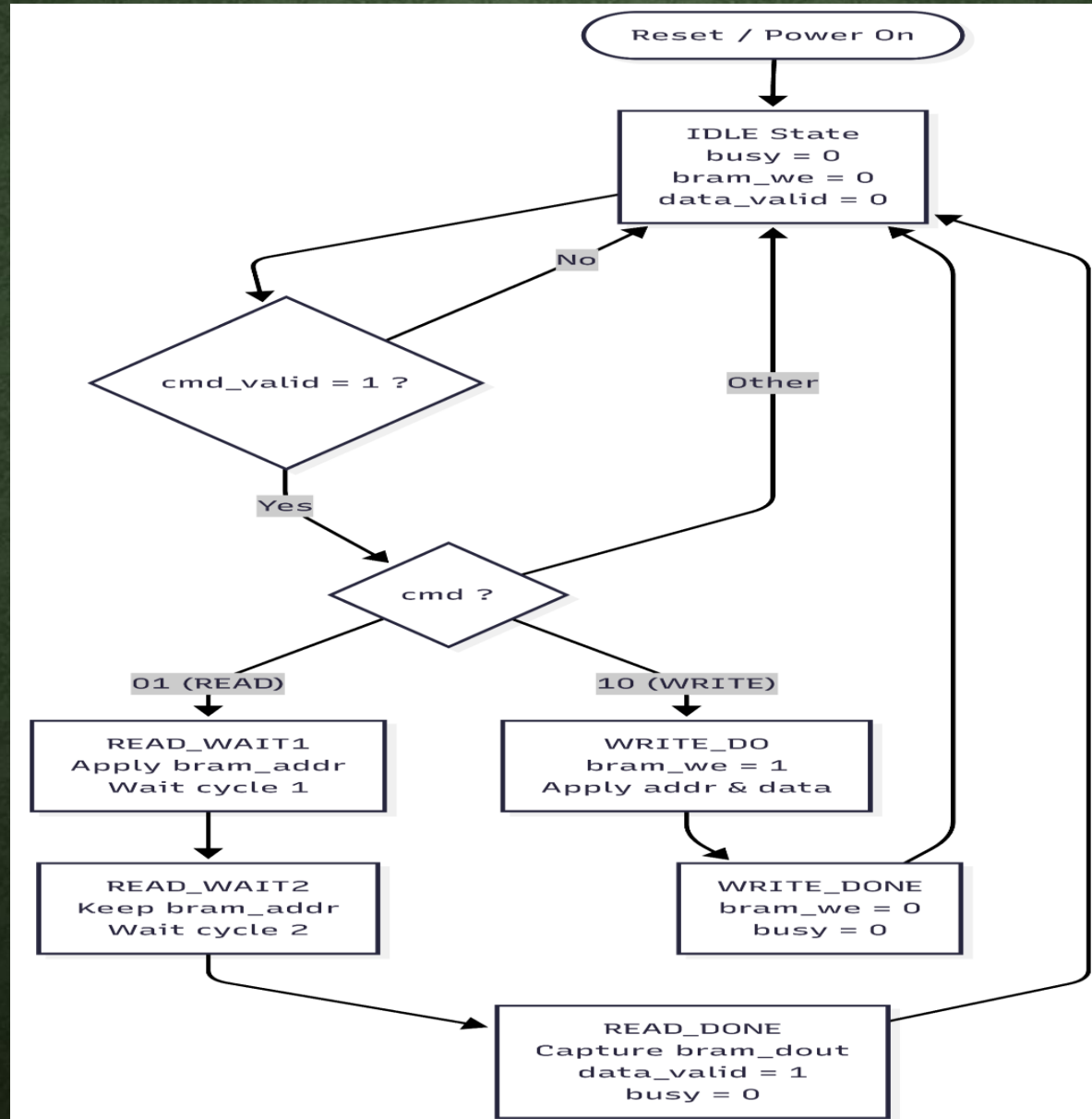
یاسین سموعی



شماتیک FSM



دیاگرام FSM



کد Top Module بخش اول

```
-- file: top_bram_fsm.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

```
entity top_bram_fsm is
  generic(
    ADDR_WIDTH : integer := 8;
    DATA_WIDTH : integer := 8
  );
  port(
    clk      : in std_logic;
    rst      : in std_logic;
    cmd      : in std_logic_vector(1 downto 0);
    cmd_valid : in std_logic;
    addr_in  : in std_logic_vector(ADDR_WIDTH-1 downto 0);
    data_in  : in std_logic_vector(DATA_WIDTH-1 downto 0);
    data_out : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_valid : out std_logic;
    busy     : out std_logic
  );
end entity;
```

```
architecture structural of top_bram_fsm is
  signal bram_we_s : std_logic;
  signal bram_addr_s : std_logic_vector(ADDR_WIDTH-1 downto 0);
  signal bram_din_s : std_logic_vector(DATA_WIDTH-1 downto 0);
  signal bram_dout_s : std_logic_vector(DATA_WIDTH-1 downto 0);

  -- Internal signals that can be probed in simulation
  signal controller_state_debug : std_logic_vector(2 downto 0);
  signal bram_we_debug : std_logic;
  signal bram_addr_debug : std_logic_vector(ADDR_WIDTH-1 downto 0);
  signal bram_din_debug : std_logic_vector(DATA_WIDTH-1 downto 0);
  signal bram_dout_debug : std_logic_vector(DATA_WIDTH-1 downto 0);
```

```
-- Component declarations
component bram_controller
  generic(
    ADDR_WIDTH : integer;
    DATA_WIDTH : integer
  );
  port(
    clk      : in std_logic;
    rst      : in std_logic;
    cmd      : in std_logic_vector(1 downto 0);
    cmd_valid : in std_logic;
    addr_in  : in std_logic_vector(ADDR_WIDTH-1 downto 0);
    data_in  : in std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_dout : in std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_we   : out std_logic;
    bram_addr : out std_logic_vector(ADDR_WIDTH-1 downto 0);
    bram_din  : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_out  : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_valid : out std_logic;
    busy      : out std_logic
  );
end component;

component bram_sram
  generic(
    ADDR_WIDTH : integer;
    DATA_WIDTH : integer
  );
  port(
    clk      : in std_logic;
    rst      : in std_logic;
    cmd      : in std_logic_vector(1 downto 0);
    cmd_valid : in std_logic;
    addr_in  : in std_logic_vector(ADDR_WIDTH-1 downto 0);
    data_in  : in std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_dout : in std_logic_vector(DATA_WIDTH-1 downto 0);
    bram_we   : out std_logic;
    bram_addr : out std_logic_vector(ADDR_WIDTH-1 downto 0);
    bram_din  : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_out  : out std_logic_vector(DATA_WIDTH-1 downto 0);
    data_valid : out std_logic;
    busy      : out std_logic
  );
end component;
```

یاسین سموعی

کد Top Module بخش دوم

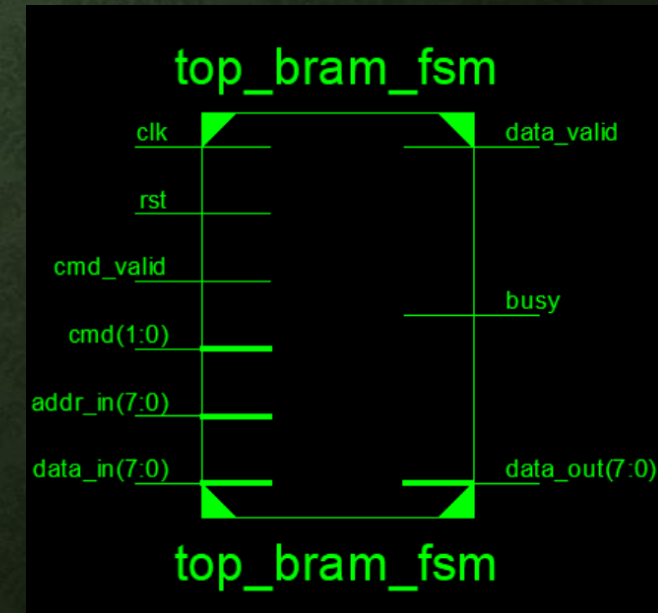
```
port(  
    clk :in std_logic;  
    rst :in std_logic;  
    we  :in std_logic;  
    addr:in std_logic_vector(ADDR_WIDTH-1 downto 0);  
    din :in std_logic_vector(DATA_WIDTH-1 downto 0);  
    dout :out std_logic_vector(DATA_WIDTH-1 downto 0)  
);  
end component;
```

```
begin  
    -- Connect debug signals  
    bram_we_debug  <= bram_we_s;  
    bram_addr_debug <= bram_addr_s;  
    bram_din_debug  <= bram_din_s;  
    bram_dout_debug <= bram_dout_s;
```

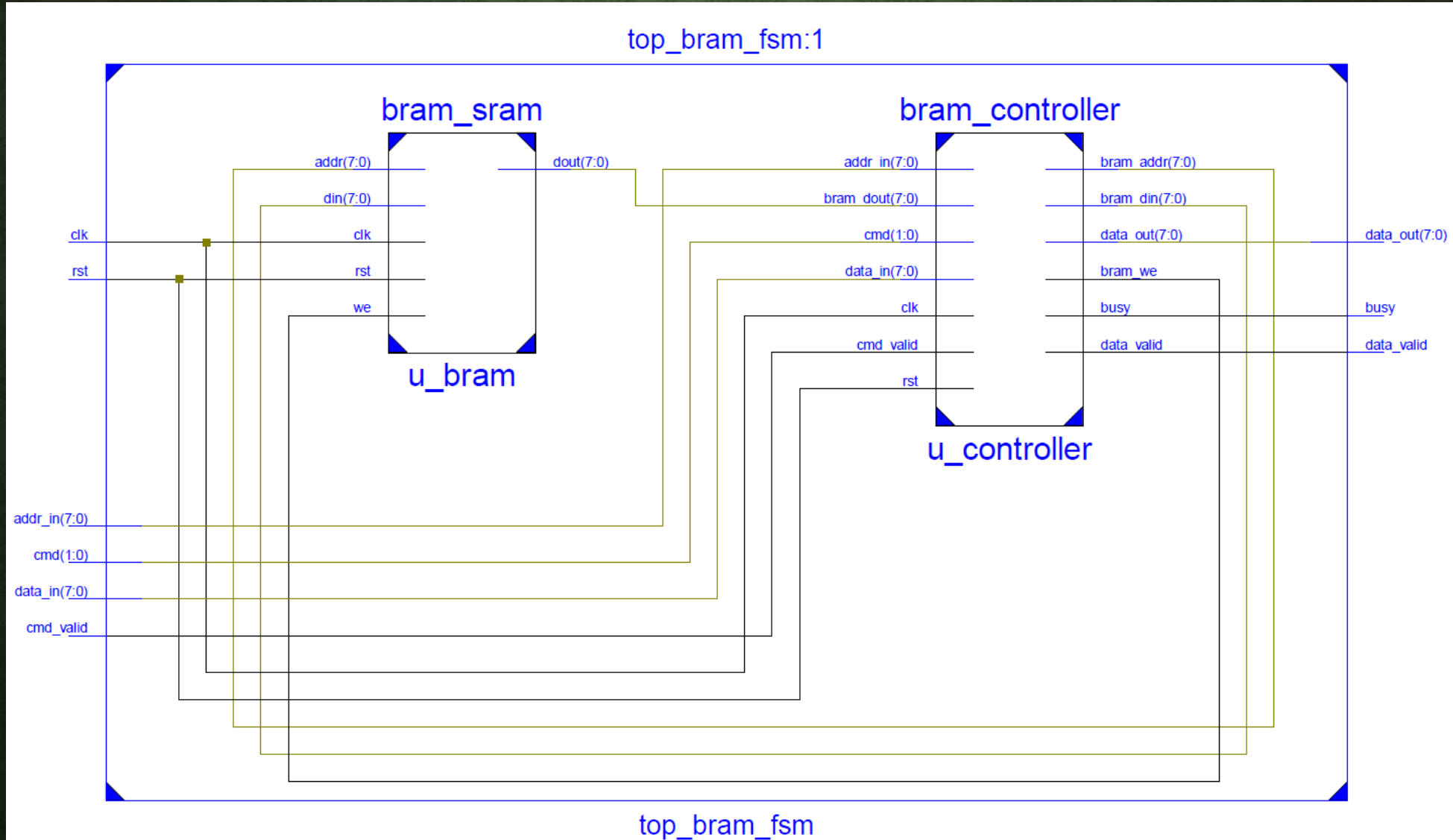
```
-- Instantiate controller  
u_controller: bram_controller  
    generic map (  
        ADDR_WIDTH => ADDR_WIDTH,  
        DATA_WIDTH => DATA_WIDTH  
    )  
    port map (  
        clk    => clk,  
        rst    => rst,  
        cmd    => cmd,  
        cmd_valid => cmd_valid,  
        addr_in => addr_in,  
        data_in => data_in,  
        bram_dout => bram_dout_s,  
        bram_we  => bram_we_s,  
        bram_addr => bram_addr_s,  
        bram_din => bram_din_s,  
        data_out => data_out,  
        data_valid => data_valid,  
        busy    => busy  
    );
```

```
-- Instantiate BRAM  
u_bram: bram_sram  
    generic map (  
        ADDR_WIDTH => ADDR_WIDTH,  
        DATA_WIDTH => DATA_WIDTH  
    )  
    port map (  
        clk => clk,  
        rst => rst,  
        we  => bram_we_s,  
        addr => bram_addr_s,  
        din  => bram_din_s,  
        dout => bram_dout_s  
    );
```

end architecture



ساختار Top Module



کد Test Bench بخش اول

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_bram_fsm_corrected is
end entity;
```

```
architecture sim of tb_bram_fsm_corrected is
    constant ADDR_WIDTH : integer := 8;
    constant DATA_WIDTH : integer := 8;
    constant CLK_PERIOD : time := 10 ns;
```

```
    signal clk      : std_logic := '0';
    signal rst      : std_logic := '0';
    signal cmd       : std_logic_vector(1 downto 0) := "00";
    signal cmd_valid : std_logic := '0';
    signal addr_in   : std_logic_vector(ADDR_WIDTH-1 downto 0) := (others => '0');
    signal data_in   : std_logic_vector(DATA_WIDTH-1 downto 0) := (others => '0');
    signal data_out  : std_logic_vector(DATA_WIDTH-1 downto 0);
    signal data_valid: std_logic;
    signal busy      : std_logic;
```

```
    signal sim_done : boolean := false;
```

```
begin
    uut: entity work.top_bram_fsm
        generic map (
            ADDR_WIDTH => ADDR_WIDTH,
            DATA_WIDTH => DATA_WIDTH
        )
        port map (
            clk      => clk,
            rst      => rst,
            cmd       => cmd,
            cmd_valid => cmd_valid,
            addr_in   => addr_in,
            data_in   => data_in,
            data_out  => data_out,
            data_valid => data_valid,
            busy      => busy
        );
```

-- Clock generation

```
clk_proc: process
begin
    while not sim_done loop
        clk <= '0';
        wait for CLK_PERIOD/2;
        clk <= '1';
        wait for CLK_PERIOD/2;
    end loop;
    wait;
end process;
```

کد Test Bench بخش دوم

```
-- Detailed monitor
monitor_proc: process(clk)
  variable cycle : integer := 0;
begin
  if rising_edge(clk) then
    cycle := cycle + 1;
    report "Cycle " & integer'image(cycle) &
      ": Cmd=" & integer'image(to_integer(unsigned(cmd))) &
      " CmdV=" & std_logic'image(cmd_valid) &
      " Addr=0x" & integer'image(to_integer(unsigned(addr_in))) &
      " Din=0x" & integer'image(to_integer(unsigned(data_in))) &
      " Busy=" & std_logic'image(busy) &
      " DValid=" & std_logic'image(data_valid) &
      " Dout=0x" & integer'image(to_integer(unsigned(data_out)));
  end if;
end process;
```

```
-- Corrected stimulus
stim_proc: process
  variable test_passed : boolean := true;
begin
  report "=== Starting Corrected Test ===";

  -- Reset
  rst <= '1';
  wait for 25 ns;
  wait until rising_edge(clk);
  rst <= '0';
  wait for 20 ns;

  -----
  -- Test 1: Write 0xAA to address 0x01
  -----

  report "Test 1: Write 0xAA to address 0x01";
  wait until rising_edge(clk);
  addr_in <= x"01";
  data_in <= x"AA";
  cmd <= "10"; -- Write command
  cmd_valid <= '1';
  wait until rising_edge(clk);
  cmd_valid <= '0';
  cmd <= "00";

  -- Wait for write to complete (2 cycles based on log)
  wait until busy = '0';
  wait for 20 ns;
```

کد Test Bench بخش سوم

```
-----  
-- Test 2: Read from address 0x01 (should be AA)  
-----
```

```
report "Test 2: Read from address 0x01 (expect 0xAA)";  
wait until rising_edge(clk);  
addr_in  <= x"01";  
data_in  <= x"00"; -- Don't care for read  
cmd      <= "01"; -- Read command  
cmd_valid <= '1';  
wait until rising_edge(clk);  
cmd_valid <= '0';  
cmd      <= "00";
```

```
-- Wait for data_valid with proper timing (3 cycles based on log)  
for i in 1 to 5 loop  
    wait until rising_edge(clk);  
    exit when data_valid = '1';  
end loop;
```

```
-- Check result  
if data_valid = '1' then  
    if data_out = x"AA" then  
        report "SUCCESS: Read 0xAA from address 0x01";  
    else  
        report "FAIL: Read 0x" & integer'image(to_integer(unsigned(data_out))) &  
            " from address 0x01 (expected 0xAA)" severity error;  
        test_passed := false;  
    end if;  
else  
    report "FAIL: No data_valid received for read from address 0x01" severity error;  
    test_passed := false;  
end if;
```

```
-- Wait for operation to fully complete  
wait until busy = '0';  
wait for 20 ns;
```

```
-----  
-- Test 3: Write 0x55 to address 0x02  
-----
```

```
report "Test 3: Write 0x55 to address 0x02";  
wait until rising_edge(clk);  
addr_in  <= x"02";  
data_in  <= x"55";  
cmd      <= "10";  
cmd_valid <= '1';  
wait until rising_edge(clk);  
cmd_valid <= '0';  
cmd      <= "00";
```

```
wait until busy = '0';  
wait for 20 ns;
```

```
-----  
-- Test 4: Read from address 0x02 (should be 55)  
-----
```

```
report "Test 4: Read from address 0x02 (expect 0x55)";  
wait until rising_edge(clk);  
addr_in  <= x"02";  
data_in  <= x"00";  
cmd      <= "01";  
cmd_valid <= '1';  
wait until rising_edge(clk);  
cmd_valid <= '0';  
cmd      <= "00";
```

```
-- Wait for data_valid  
for i in 1 to 5 loop  
    wait until rising_edge(clk);  
    exit when data_valid = '1';  
end loop;
```

```
-- Check result  
if data_valid = '1' then  
    if data_out = x"55" then  
        report "SUCCESS: Read 0x55 from address 0x02";  
    else  
        report "FAIL: Read 0x" &  
            integer'image(to_integer(unsigned(data_out))) &  
            " from address 0x02 (expected 0x55)" severity error;  
        test_passed := false;  
    end if;  
else  
    report "FAIL: No data_valid received for read from address  
0x02" severity error;  
    test_passed := false;  
end if;
```

کد Test Bench بخش چهارم

```
-----  
-- Additional test: Write and read from address 0x03  
-----
```

```
report "Test 5: Write 0x33 to address 0x03 and read back";  
wait until rising_edge(clk);  
addr_in  <= x"03";  
data_in  <= x"33";  
cmd      <= "10";  
cmd_valid <= '1';  
wait until rising_edge(clk);  
cmd_valid <= '0';  
cmd      <= "00";
```

```
wait until busy = '0';  
wait for 20 ns;
```

```
-- Read back  
wait until rising_edge(clk);  
addr_in  <= x"03";  
data_in  <= x"00";  
cmd      <= "01";  
cmd_valid <= '1';  
wait until rising_edge(clk);  
cmd_valid <= '0';  
cmd      <= "00";
```

```
for i in 1 to 5 loop  
  wait until rising_edge(clk);  
  exit when data_valid = '1';  
end loop;
```

```
if data_valid = '1' then  
  if data_out = x"33" then  
    report "SUCCESS: Read back 0x33 from address 0x03";  
  else  
    report "FAIL: Read back 0x" & integer'image(to_integer(unsigned(data_out))) &  
      " from address 0x03 (expected 0x33)" severity error;  
    test_passed := false;  
  end if;  
else  
  report "FAIL: No data_valid received for read from address 0x03" severity error;  
  test_passed := false;  
end if;
```

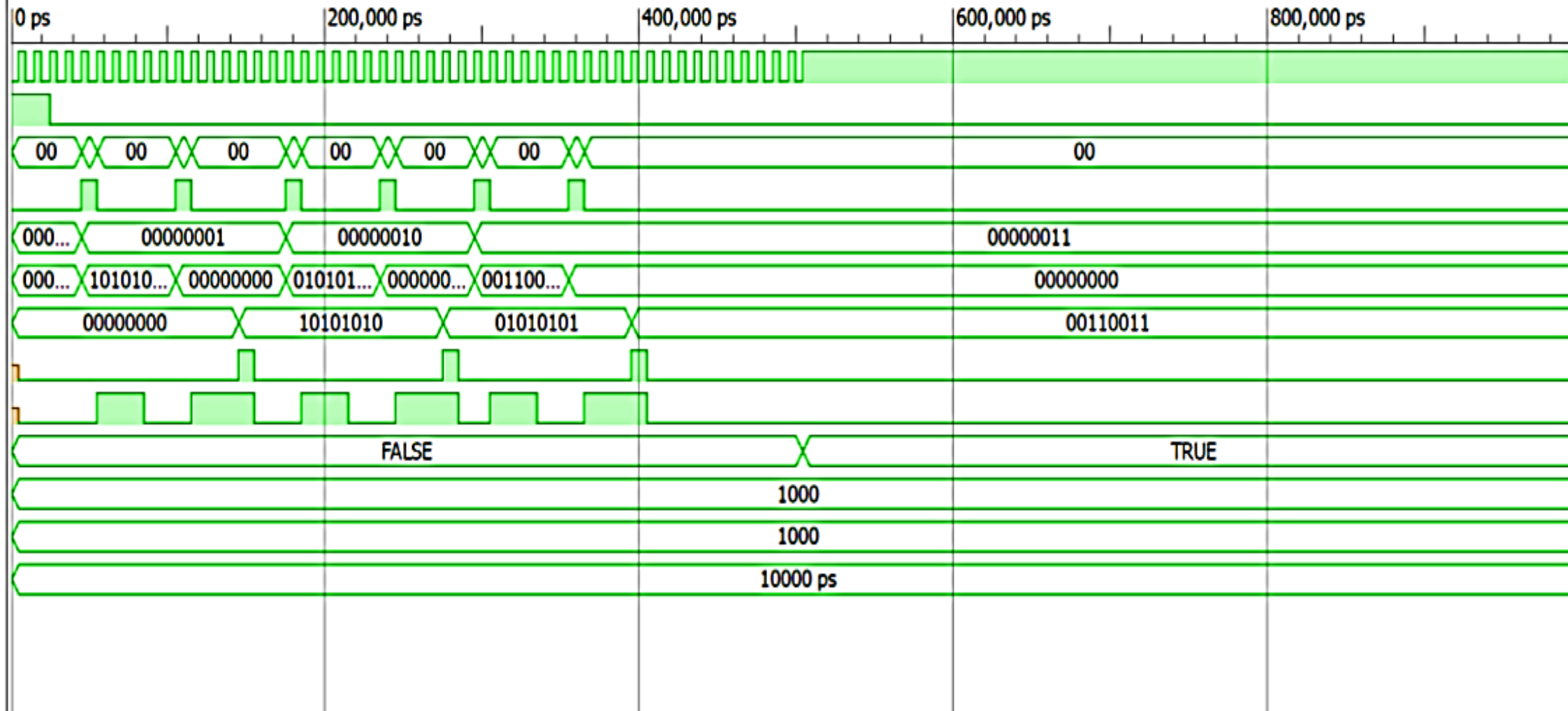
```
-----  
-- Final summary  
-----
```

```
report "=====";  
if test_passed then  
  report "ALL TESTS PASSED!";  
else  
  report "SOME TESTS FAILED!";  
end if;  
report "=====";
```

```
wait for 100 ns;  
sim_done <= true;  
wait;  
end process;
```

```
end architecture;
```

خروجی شبیه سازی



تحلیل شبیه سازی (بخش اول)

الف) عملیات نوشتن (Write)

- مدت زمان واقعی: ۴ سیکل کلاک
- تحلیل حالت‌ها بر اساس لاگ شبیه‌سازی:

سیکل اول: (IDLE → WRITE_DO)

- با فعال شدن cmd_valid و cmd="10"، کنترلر بلافاصله سیگنال‌های bram_addr، bram_din و bram_we='1' را اعمال می‌کند.
- سیگنال busy فعال می‌شود.

سیکل دوم: (WRITE_DO → WRITE_DONE)

- bram_we همچنان فعال باقی می‌ماند.
- FSM به حالت WRITE_DONE منتقل می‌شود.

سیکل سوم: (WRITE_DONE → WRITE_DONE)

- در این سیکل اضافی، bram_we غیرفعال می‌شود اما busy همچنان فعال است.
- این تأخیر اضافی ناشی از طراحی FSM است.

سیکل چهارم: (WRITE_DONE → IDLE)

- کنترلر به حالت IDLE باز می‌گردد و سیگنال busy غیرفعال می‌شود.

تحلیل شبیه سازی (بخش دوم)

ب) عملیات خواندن (Read)

- مدت زمان واقعی: ۵ سیکل کلاک

تحلیل حالت‌ها بر اساس لاگ شبیه‌سازی:

سیکل اول: (IDLE → READ_WAIT1)

- با فعال شدن cmd_valid و cmd="01"، آدرس به BRAM اعمال می‌شود.
- سیگنال busy فعال می‌شود.

سیکل دوم: (READ_WAIT1 → READ_WAIT2)

- BRAM بر اساس آدرس سیکل قبل، داده را از حافظه واکنشی می‌کند latency یک سیکلی

سیکل سوم: (READ_WAIT2 → READ_DONE)

- داده از خروجی BRAM در دسترس است.
- FSM به حالت READ_DONE منتقل می‌شود.

سیکل چهارم: (READ_DONE → READ_DONE)

- سیگنال data_valid فعال می‌شود.
- داده از bram_dout به data_out_reg منتقل می‌شود.
- busy همچنان فعال است.

سیکل پنجم: (READ_DONE → IDLE)

- کنترلر به حالت IDLE باز می‌گردد.
- سیگنال busy غیرفعال می‌شود.

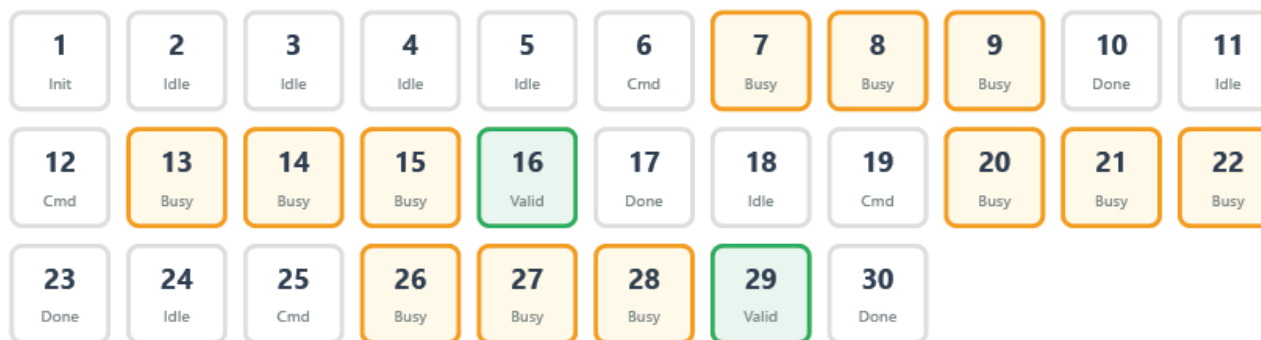


Detailed Test Results

Test #	Description	Operation	Address	Data	Cycles	Duration	Status
1	Write 0xAA to address 0x01	WRITE	0x01	0xAA (0x170)	6-10	50 ns (5 cycles)	PASSED
2	Read from address 0x01	READ	0x01	0xAA (0x170)	12-17	60 ns (6 cycles)	PASSED
3	Write 0x55 to address 0x02	WRITE	0x02	0x55 (0x85)	19-23	50 ns (5 cycles)	PASSED
4	Read from address 0x02	READ	0x02	0x55 (0x85)	25-30	60 ns (6 cycles)	PASSED
5	Write 0x33 and read back from 0x03	WRITE+READ	0x03	0x33 (0x51)	31-41	110 ns (11 cycles)	PASSED



Operation Timeline (First 30 Cycles)



Idle/Command



Busy



Data Valid